Software Dependability

Software Fault Tolerance and Assessment

The Fundamental Principle of Dependability

“If all faults to which a system is subject are identified and their effects are properly mitigated, the system will meet its dependability requirements.”

There are different ways to mitigate faults; they can be

- **Avoided**: If we build systems without faults, we do not need to do anything else.
- **Eliminated**: Do you remember software testing?
- **Tolerated**: Managed in a way that they do not cause failures.
- **Forecast**: Suppose when their effects will occur and be prepared for them.

Since not all projects are the same and perfection is not possible, we need to address dependability issues.

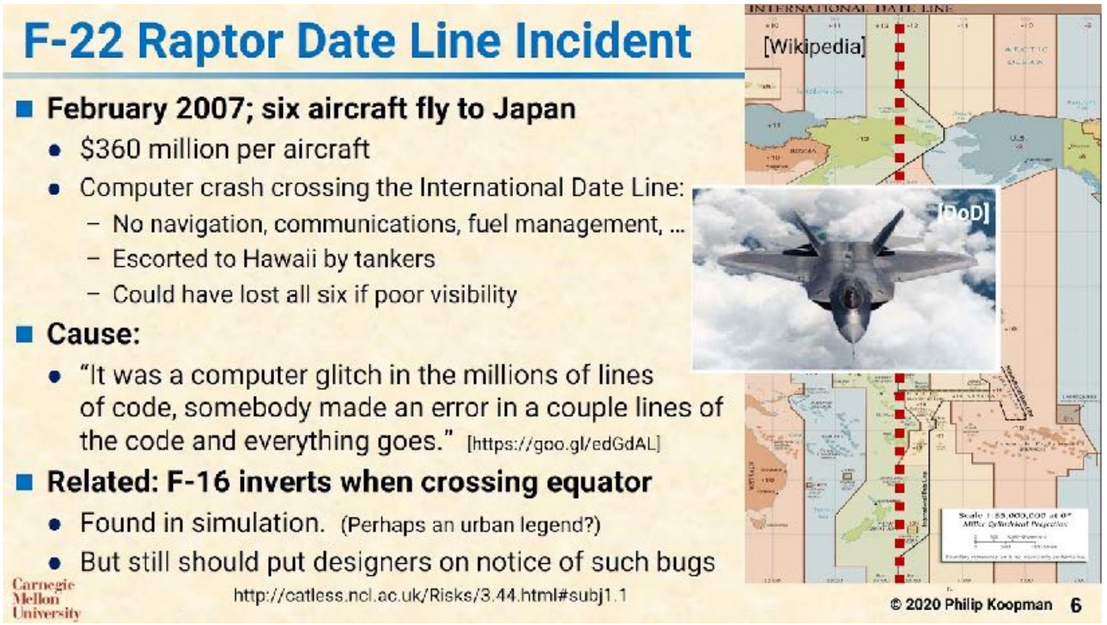
This week, we see how faults can be tolerated and assessed.

Design Faults are Hard to Tolerate

Design faults are way harder to anticipate than degradation faults.
You can estimate when you change a hard drive, not when you refactor a class.

The effects of design faults are hard to predict. A bug of a couple of lines can blind an F-22.

The failure semantics components exhibit are hard to define.
Consider you have software; will it fail if a bug manifests itself, and will it terminate?



Could you modify your software to automatically detect erroneous states, revert to the state before the error occurred, correct the error, and then continue execution?

Self-Healing Systems and Error Recovery

Self-healing systems automatically detect, diagnose, and fix their issues.
They often use artificial intelligence to keep operating with little to no human intervention.

Error recovery is crucial to ensure a system can continue operating correctly even when faults occur.

Forward error recovery attempts to bring the system to a new, correct state from the erroneous state, without rolling back to a previous state.

1. The system detects an error.
2. It analyzes the current (possibly corrupted) state.
3. It tries to correct the state or adapt the computation so that it can continue safely.

Forward error recovery is used when rollback is not feasible (e.g., in real-time systems) or in systems with built-in redundancy or self-healing capabilities. For example, in a spacecraft control system, if a sensor fails, the system might switch to a backup sensor and continue operation without reverting to a previous state.

Backward error recovery restores the system to a previously known correct state, essentially "undoing" the effects of the error.

1. The system periodically saves snapshots (checkpoints) of its state.
2. Upon detecting an error, it rolls back to the last known good checkpoint.
3. The computation resumes from that point.

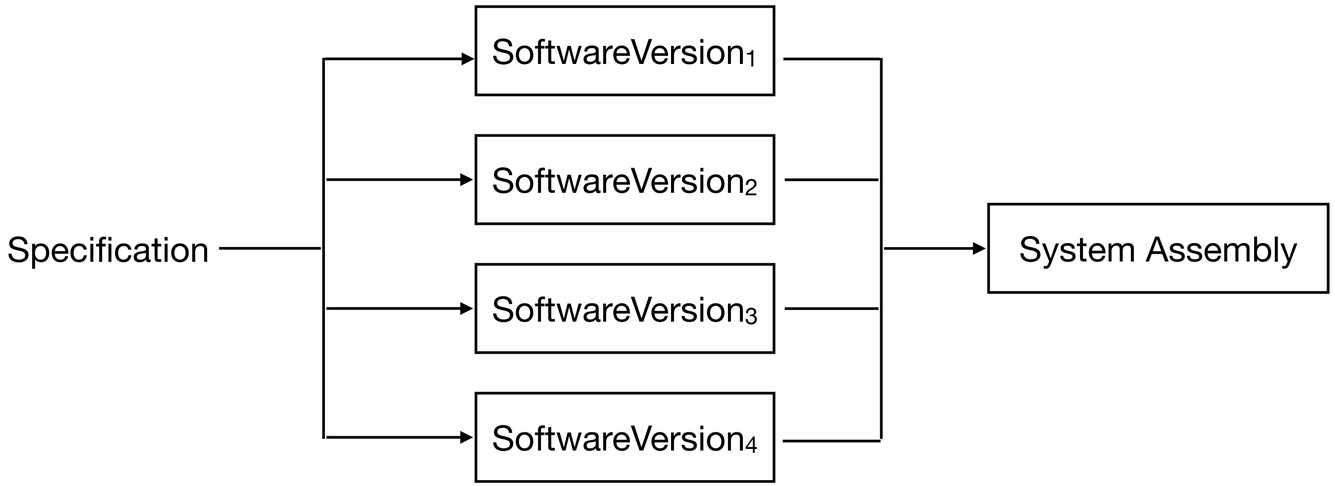
Backward error recovery is used in systems where states can be saved and restored efficiently, and in applications where correctness matters more than real-time performance. For example, in a database system, if a transaction fails, the system rolls back to the state it was in before the transaction began.

Design Diversity

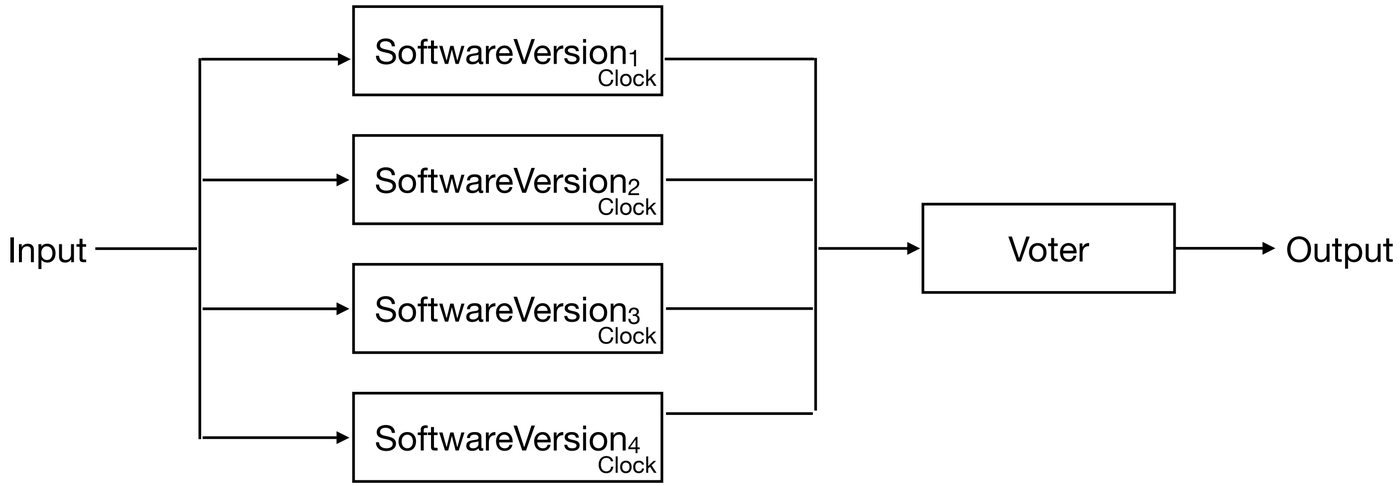
If software systems are built with different designs to provide the same functionality, they might not fail on the same input. Software systems could be diverse if developed:

- by different development teams
- using different software development tools
- using different software design techniques
- in different periods.

To be really diverse, all these versions should be developed in *isolation*.



Design Diversity: N-Version Systems



Let's assume we have N-modular software redundancy.
If the software components are just copies, if one fails, they will all fail, which is correct if:

- Each copy receives the same data.
- Software is deterministic.

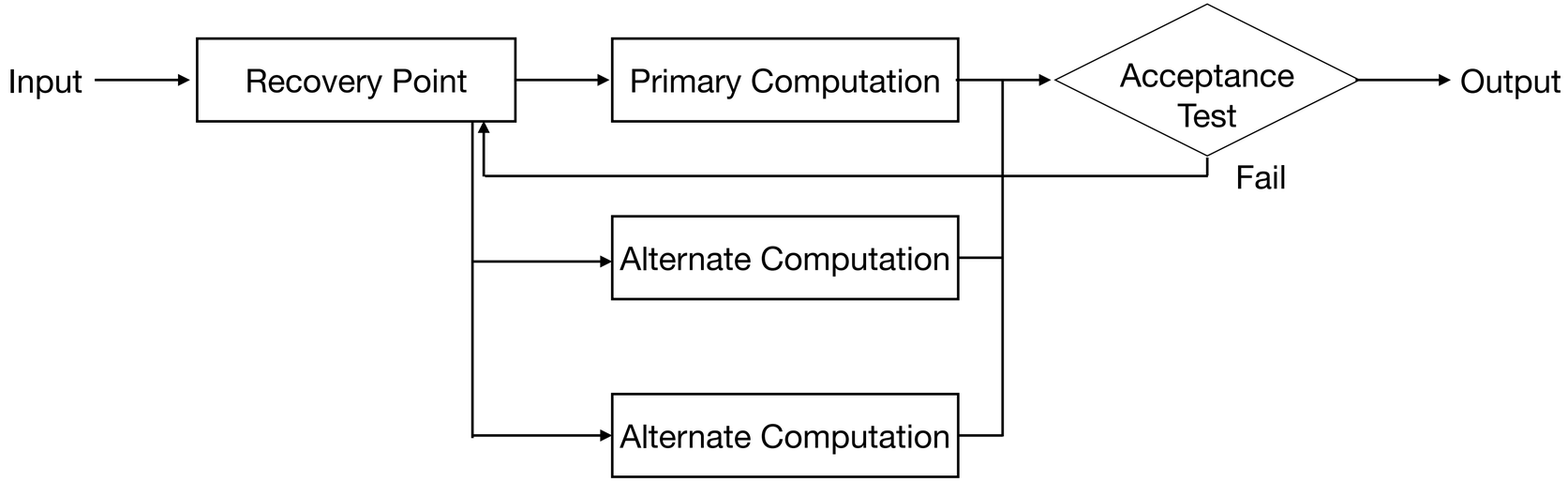
In practice, it is easy to violate such conditions.

N different versions run in parallel, and a voter collects their output and decides the final result, for example leveraging majority voting.
The clock synchronizes the execution of the different versions and the voter, allowing the voter to decide whether some versions fail.

The biggest benefits appear with *Heisenbugs*.

The n-version architecture implements forward error recovery.

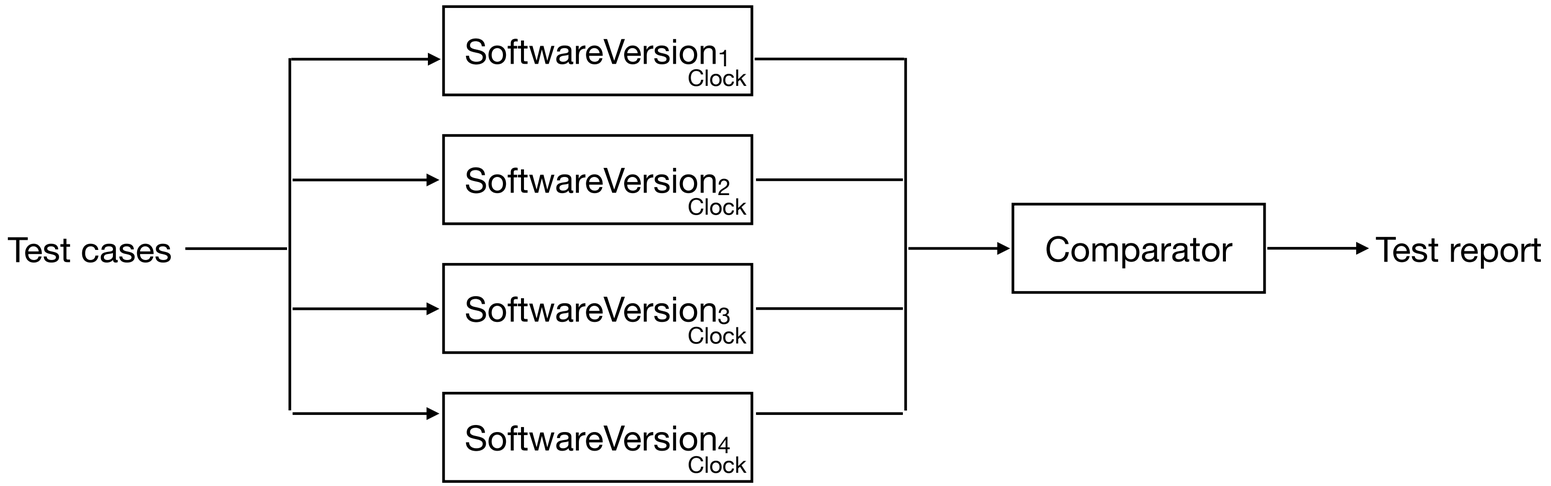
Design Diversity: Recovery Blocks



In a recovery block, the versions, also known as alternates, run sequentially, with the first labeled as the primary. If the acceptance test passes on the primary computation, the recovery block stops; otherwise, the system restores and runs an alternate.

Unlike the N-version architecture, a recovery block cannot mask faults because running the alternate causes a delay. The recovery block architecture implements backward error recovery.

Comparison Checking



Four possible outcomes:

1. All versions produce the correct output: yeah!
2. Some versions produce correct output. Developers could use the info to fix the failing versions.
3. All versions produce incorrect but different results.
   Using the differences between the failures, developers can try to address the bugs.
4. All versions produce incorrect but identical outputs.
   The comparator will report no issues, and developers will be in trouble.

Options 2 and 3 could lead to false positives. Option 4 is a nightmare. However, life is much worse than it appears.

The Consistent Comparison Problem

The Consistent Comparison Problem (CCP) arises when multiple independently developed software versions, as in N-version programming, produce inconsistent results because they compare finite-precision numbers differently. Even if all versions are correct and receive the same input, they might:

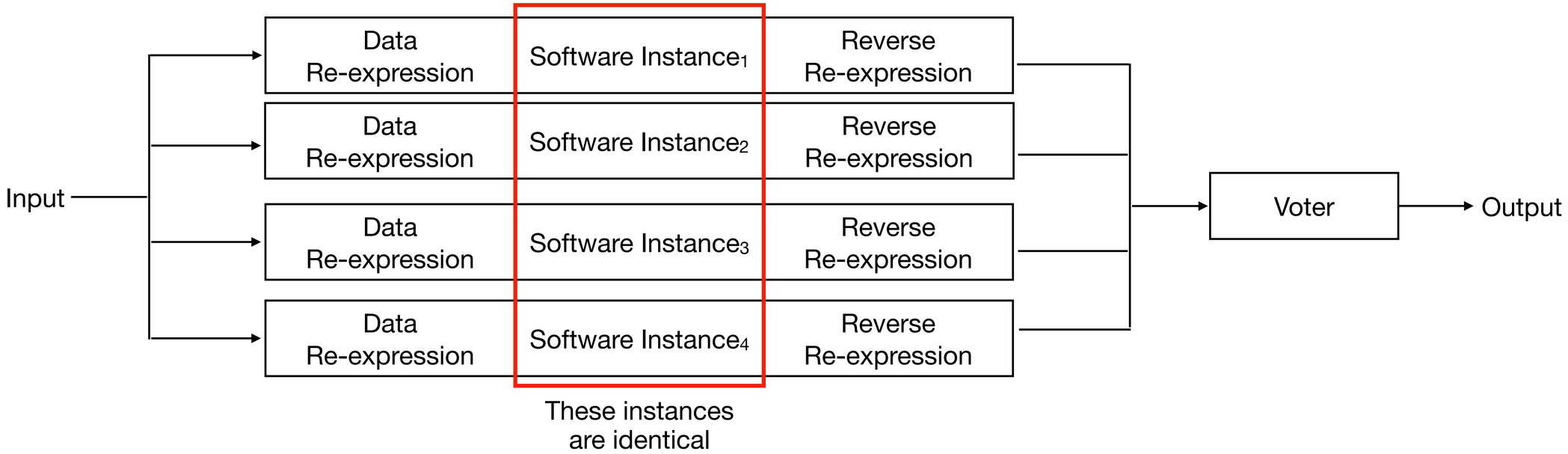
- Use different algorithms,
- Execute operations in different orders,
- Run on different hardware with slightly different floating-point behavior.

As a result, comparisons like $x < y$ may result in different boolean results across versions, leading to diverging control flows and ultimately different outputs, even though no version has failed.

The only way to solve this problem is through communication, but that also violates the *isolation* constraint.

Data Diversity: N-Copy Systems

What would happen if we ran multiple instances of the same implementation with different data?



An exact reexpression algorithm changes the input data so that the reexpressed data produces the same output as the original data through algorithmic means.
An inexact reexpression algorithm modifies the input data for a program so that the output generated from the reexpressed data remains acceptable to the application.

Developing the voter is easier because the CCP cannot happen. The number of instances is not easy to predict. However, scaling up is easy and not very expensive.

Dependability Assessment: Prescriptive Standards

Prescriptive standards are formal guidelines and requirements that specify how software systems should be developed, verified, and maintained to ensure they are reliable, safe, secure, and robust.

They are especially important in safety-critical fields like aerospace, automotive, healthcare, and nuclear systems, where software failures can have serious consequences.

Prescriptive Standards in Practice!

Let’s start with a raffle!!!



This is Charmender, who studies how to make blockchain smart contracts more dependable.



This is Squirtle, who studies software engineering issues in AI-enabled systems.



This is Bulbasaur, who studies how to make advanced driver assistance systems more dependable.

You need to select one of them for the next step

Prescriptive Standards in Practice!



If you selected Charmender, you will read the OWASP Smart Contract Security Verification Standard



If you selected Squirtle, you will read the Assessment List for Trustworthy Artificial Intelligence (ALTAI)



If you selected Bulbasaur, you will read SAE J3016_202104 Taxonomy and Definitions for Terms Related to Driving Automation Systems for On-Road Motor Vehicles


Break